

DEFINITION OF TERMS

This specification uses a particular set of terminology, defined in this section. This section has three parts:

General ACPI terms are defined and presented alphabetically.

The ACPI global system states (working, sleeping, soft off, and mechanical off) are defined. Global system states apply to the entire system, and are visible to the user.

The ACPI device power states are defined. Device power states are states of particular devices; as such, they are generally not visible to the user. For example, some devices may be in the off state even though the system as a whole is in the working state. Device states apply to any device on any bus.

2.1 General ACPI Terminology

Advanced Configuration and Power Interface (ACPI)

As defined in this document, ACPI is a method for describing hardware interfaces in terms abstract enough to allow flexible and innovative hardware implementations and concrete enough to allow shrink-wrap OS code to use such hardware interfaces.

ACPI Hardware

Computer hardware with the features necessary to support OSPM and with the interfaces to those features described using the Description Tables as specified by this document.

ACPI Namespace

A hierarchical tree structure in OS-controlled memory that contains named objects. These objects may be data objects, control method objects, bus/device package objects, and so on. The OS dynamically changes the contents of the namespace at run-time by loading definition blocks from the ACPI Tables that reside in the ACPI system firmware. All the information in the ACPI Namespace comes from the Differentiated System Description Table (DSDT), which contains the Differentiated Definition Block, and one or more other definition blocks.

ACPI Machine Language (AML)

Pseudo-code for a virtual machine supported by an ACPI-compatible OS and in which ACPI control methods and objects are written. The AML encoding definition is provided in section 19, “ACPI Machine Language (AML) Specification.”

Add-in Card

A generic term used to refer to any device which can be inserted or removed from a platform through a connection bus, such as PCI. Add-in cards are typically inserted within a platform’s physical enclosure, rather than residing physically external to a platform. An add-in card will have its own devices and associated firmware, and may have its own Expansion ROM Firmware.

Advanced Programmable Interrupt Controller (APIC)

An interrupt controller architecture commonly found on Intel Architecture-based 32-bit PC systems. The APIC

architecture supports multiprocessor interrupt management (with symmetric interrupt distribution across all processors), multiple I/O subsystem support, 8259A compatibility, and inter-processor interrupt support. The architecture consists of local APICs commonly attached directly to processors and I/O APICs commonly in chip sets.

ACPI Source Language (ASL)

The programming language equivalent for AML. ASL is compiled into AML images. The ASL statements are defined in section 18, “ACPI Source Language (ASL) Reference.”

Address Range Scrub (ARS)

Process by which regions of memory can be scrubbed to look for memory locations that contain correctable or uncorrectable errors.

BIOS

BIOS (Basic Input/Output System) is firmware that provides basic boot capabilities for a platform; it is used here to refer specifically to traditional x86 BIOS, and not as a general term for all firmware, or a replacement term for UEFI Core System BIOS. The ambiguity of this the term is what we are trying to remove. See also: *Legacy BIOS*, *System BIOS*.

Boot Firmware

Generic term to describe any firmware on a platform used during the boot process. Use a more specific term, if possible.

Component

Synonym for device. Please use the term “device” if possible.

Control Method

A control method is a definition of how the OS can perform a simple hardware task. For example, the OS invokes control methods to read the temperature of a thermal zone. Control methods are written in an encoded language called AML that can be interpreted and executed by the ACPI-compatible OS. An ACPI-compatible system must provide a minimal set of control methods in the ACPI tables. The OS provides a set of well-defined control methods that ACPI table developers can reference in their control methods. OEMs can support different revisions of chip sets with one version of platform firmware by either including control methods in the platform firmware that test configurations and respond as needed or including a different set of control methods for each chip set revision.

Central Processing Unit (CPU) or Processor

The part of a platform that executes the instructions that do the work. An ACPI-compatible OS can balance processor performance against power consumption and thermal states by manipulating the processor performance controls. The ACPI specification defines a working state, labeled G0 (S0), in which the processor executes instructions. Processor sleeping states, labeled C1 through C3, are also defined. In the sleeping states, the processor executes no instructions, thus reducing power consumption and, potentially, operating temperatures. The ACPI specification also defines processor performance states, where the processor (while in C0) executes instructions, but with lower performance and (potentially) lower power consumption and operating temperature. For more information, see [Section 8](#).

A definition block contains information about hardware implementation and configuration details in the form of data and control methods, encoded in AML. An OEM can provide one or more definition blocks in the ACPI Tables. One definition block must be provided: the Differentiated Definition Block, which describes the base system. Upon loading the Differentiated Definition Block, the OS inserts the contents of the Differentiated Definition Block into the ACPI Namespace. Other definition blocks, which the OS can dynamically insert and remove from the active ACPI Namespace, can contain references to the Differentiated Definition Block. For more information, see [Definition Blocks](#).

Device

A generic term used to refer to any computing, input/output or storage element, or any collection of computing, input/output or storage elements, on a platform. An example of a device is a CPU, APU, embedded controller (EC), BMC, Trusted Platform Module (TPM), graphics processing unit (GPU), network interface controller

(NIC), hard disk drive (HDD), solid state drive (SSD), Read Only Memory (ROM), flash ROM, or any of the large number of other possible devices. If at all possible, use a more specific term.

Device Context

The variable data held by the device; it is usually volatile. The device might forget this information when entering or leaving certain states (for more information, see *Device Power State Definitions*), in which case the OS software is responsible for saving and restoring the information. Device Context refers to small amounts of information held in device peripherals. See *System Context*.

Device Firmware

Firmware that is only used by a specific device and cannot be used with any other device. This firmware is typically provided by the device manufacturer.

Differentiated System Description Table (DSDT)

An OEM must supply a DSDT to an ACPI-compatible OS. The DSDT contains the Differentiated Definition Block, which supplies the implementation and configuration information about the base system. The OS always inserts the DSDT information into the ACPI Namespace at system boot time and never removes it.

Device Physical Address (DPA)

A Device relative memory address.

Embedded Controller

The general class of micro-controllers used to support OEM-specific supports embedded controllers in any platform design, as long as the micro-controller conforms to one of the models described in this section. The embedded controller performs complex low-level functions through a simple interface to the host microprocessor(s).

ACPI defines a standard hardware and software communications interface between an OS bus enumerator and an embedded controller. This allows any OS to provide a standard bus enumerator that can directly communicate with an embedded controller in the system, thus allowing other drivers within the system to communicate with and use the resources of system embedded controllers. This in turn enables the OEM to provide platform features that the OS and applications can use.

Embedded Controller Interface

A standard hardware and software communications interface between an OS driver and an embedded controller. This allows any OS to provide a standard driver that can directly communicate with an embedded controller in the system, thus allowing other drivers within the system to communicate with and use the resources of system embedded controllers (for example, Smart Battery and AML code). This in turn enables the OEM to provide platform features that the OS and applications can use.

Expansion ROM Firmware

Peripheral Component Interconnect (PCI) term for firmware executed on a host processor which is used by an add-in device during the boot process. This includes Option ROM Firmware and UEFI drivers. Expansion ROM Firmware may be embedded as part of the Host Processor Boot Firmware, or may be separate (e.g., from an add-in card). See also: *Option ROM Firmware*.

Firmware

Generic term to describe any BIOS or firmware on a platform; it refers to the general class of things, not a specific type. Use a more specific term, if possible.

Firmware ACPI Control Structure (FACS)

A structure in read/write memory that the platform runtime firmware uses for handshaking between the firmware and the OS. The FACS is passed to an ACPI-compatible OS via the Fixed ACPI Description Table (FADT). The FACS contains the system's hardware signature at last boot, the firmware waking vector, and the Global Lock.

Firmware Storage Device

A memory device used to store firmware. This could include Read Only Memory (ROM), flash memory, eMMC, UFS drives, etc.

Fixed ACPI Description Table (FADT)

A table that contains the ACPI Hardware Register Block implementation and configuration details that the OS

needs to directly manage the ACPI Hardware Register Blocks, as well as the physical address of the DSDT, which contains other platform implementation and configuration details. An OEM must provide an FADT to an ACPI-compatible OS in the RSDT/XSDT. The OS always inserts the namespace information defined in the Differentiated Definition Block in the DSDT into the ACPI Namespace at system boot time, and the OS never removes it.

Fixed Features

A set of features offered by an ACPI interface. The ACPI specification places restrictions on where and how the hardware programming model is generated. All fixed features, if used, are implemented as described in this specification so that OSPM can directly access the fixed feature registers.

Fixed Feature Events

A set of events that occur at the ACPI interface when a paired set of status and event bits in the fixed feature registers are set at the same time. When a fixed feature event occurs, a system control interrupt (SCI) is raised. For ACPI fixed feature events, OSPM (or an ACPI-aware driver) acts as the event handler.

Fixed Feature Registers

A set of hardware registers in fixed feature register space at specific address locations in system I/O address space. ACPI defines register blocks for fixed features (each register block gets a separate pointer from the FADT). For more information, see [ACPI Hardware Features](#).

General-Purpose Event Registers

The general-purpose event registers contain the event programming model for generic features. All general-purpose events generate SCIs.

Generic Feature

A generic feature of a platform is value-added hardware implemented through control methods and general-purpose events.

Generic Interrupt Controller (GIC)

An interrupt controller architecture for ARM processor-based systems.

Global System Status

Global system states apply to the entire system, and are visible to the user. The various global system states are labeled G0 through G3 in the ACPI specification. For more information, see [Global System State Definitions](#).

Host Processor

A host processor is the primary processing unit in a platform, traditionally called a Central Processing Unit (CPU), now also sometimes referred to as an Application Processing Unit (APU), or a System on Chip (SoC). This is the processing unit on which the primary operating system (and/or hypervisor), as well as user applications run. This is the processor that is responsible for loading and executing the Host Processor Boot Firmware. This term and “Boot Processor” should be considered synonyms for this particular text clean-up effort (i.e., making them consistent should probably be part of a different ECR, if needed).

Host Processor Boot Firmware

Generic term used to describe firmware loaded and executed by the Host Processor which provides basic boot capabilities for a platform. This class of firmware is a reference to Legacy BIOS and UEFI, which were sometimes referred to as System BIOS. Where the distinction between Legacy BIOS and UEFI is not important, the term Host Processor Boot Firmware will be used. Where the distinction is important, it will be referenced appropriately. Expansion ROM firmware may also be considered as part of the Host Processor Boot Firmware. Expansion ROM Firmware may be embedded as part of the Host Processor Boot Firmware, or may be separate from the Host Processor Boot Firmware (e.g., loaded from an add-in card).

Host Processor Runtime Firmware

Host processor runtime firmware is any runtime firmware which executes on the host processor.

Ignored Bits

Some unused bits in ACPI hardware registers are designated as “ignored” in the ACPI specification. Ignored

bits are undefined and can return zero or one (in contrast to reserved bits, which always return zero). Software ignores ignored bits in ACPI hardware registers on reads and preserves ignored bits on writes.

Intel Architecture-Personal Computer (IA-PC)

A general descriptive term for computers built with processors conforming to the architecture defined by the Intel processor family based on the Intel Architecture instruction set and having an industry-standard PC architecture.

I/O APIC

An Input/Output Advanced Programmable Interrupt Controller routes interrupts from devices to the processor's local APIC.

I/O SAPIC

An Input/Output Streamlined Advanced Programmable Interrupt Controller routes interrupts from devices to the processor's local APIC.

Label Storage Area

A persistent storage area reserved for Label storage.

Legacy

A computer state where power management policy decisions are made by the platform hardware/firmware shipped with the system. The legacy power management features found in today's systems are used to support power management in a system that uses a legacy OS that does not support the OS-directed power management architecture.

Legacy BIOS

One form of Host Processor Boot Firmware used on x86 platforms which uses a legacy x86 BIOS structure. This form of host processor boot firmware has been or is being replaced by UEFI. This term will likely be most useful in distinguishing and comparing older forms of firmware to newer forms (e.g., "it was done this way in legacy BIOS, but is now done another way in UEFI). See also: *BIOS*, *System BIOS*.

Legacy Hardware

A computer system that has no ACPI or OSPM power management support.

Legacy OS

An OS that is not aware of and does not direct the power management functions of the system. Included in this category are operating systems with APM 1.x support.

Local APIC

A local Advanced Programmable Interrupt Controller receives interrupts from the I/O APIC.

Local SAPIC

A local Streamlined Advanced Programmable Interrupt Controller receives interrupts from the I/O SAPIC.

Management Firmware

Firmware used only by a Baseboard Management Controller (BMC) or other Out-of-Band (OOB) management controller.

Multiple APIC Description Table (MADT)

The Multiple APIC Description Table (MADT) is used on systems supporting the APIC and SAPIC to describe the APIC implementation. Following the MADT is a list of APIC/SAPIC structures that declare the APIC/SAPIC features of the machine.

Namespace

A namespace defines a contiguously-addressed range of Non-Volatile Memory, conceptually similar to a SCSI Logical Unit (LUN) or an NVM Express namespace. A namespace can be described by one or more Labels.

Non-Host Processor

A non-host processor is a generic term used to describe any processing unit on a platform which is not a host processor (e.g. a microcontroller, co-processor, etc). For the purposes of this particular ECR, this should also be considered a synonym for "secondary processor", those CPUs that might be on an SoC, for example, that are not the host (or "boot") processor.

NVDIMM

Non Volatile Dual In-line Memory Module.

Object

The nodes of the ACPI Namespace are objects inserted in the tree by the OS using the information in the system definition tables. These objects can be data objects, package objects, control method objects, and so on. Package objects refer to other objects. Objects also have type, size, and relative name.

Object name

Part of the ACPI Namespace. There is a set of rules for naming objects.

Operating System-directed Power Management (OSPM)

A model of power (and system) management in which the OS plays a central role and uses global information to optimize system behavior for the task at hand.

Option ROM Firmware

Legacy term for boot firmware typically executed on a host processor which is used by a device during the boot process. Option ROM firmware may be included with the host processor boot firmware or may be carried separately by a device (such as an add-in card). See also: Expansion ROM Firmware

Package

An array of objects.

Peripheral

A peripheral (also known as an external device) is a device which resides physically external to a platform and is connected to a platform, either wired or wirelessly. A peripheral is comprised of its own devices which may have their own firmware.

Persistent Memory (pmem)

Byte-addressable memory that retains its contents across power loss.

Platform

A platform consists of multiple devices assembled and working together to deliver a specific computing function, but does not include any other software other than the firmware as part of the devices in the platform. Examples of platforms include a notebook, a desktop, a server, a network switch, a blade, etc. - all without and independent of any operating system, user applications, or user data.

Platform Boot Firmware

The collection of all boot firmware on a platform. This firmware is initially loaded by a platform (such as an SoC, a motherboard, or a complete system) at power-on to do basic initialization of the platform hardware and then hand control to a boot loader or OS. In some cases this will be x86 BIOS, or it may be UEFI Core System BIOS, or it could be something else entirely. Once control has been handed over to a boot loader or an OS, this firmware has no further role.

Platform Runtime Firmware

The collection of all run-time firmware on a platform. This is firmware that can provide functions that can be invoked by an OS, but those functions are still concerned only with the platform hardware (e.g., PSCI on ARM). The assumption is that platform boot firmware has since been superseded by the OS since the OS is now up and running, but that there is still a need for an OS to access specific features of hardware that may only be possible via firmware.

Platform Firmware

The collection of platform boot firmware and platform runtime firmware.

Power Button

A user push button or other switch contact device that switches the system from the sleeping/soft off state to the working state, and signals the OS to transition to a sleeping/soft off state from the working state.

Power Management

Mechanisms in software and hardware to minimize system power consumption, manage system thermal limits,

and maximize system battery life. Power management involves trade-offs among system speed, noise, battery life, processing speed, and alternating current (AC) power consumption. Power management is required for some system functions, such as appliance (for example, answering machine, furnace control) operations.

Power Resources

Resources (for example, power planes and clock sources) that a device requires to operate in a given power state.

Power Sources

The battery (including a UPS battery) and AC line powered adapters or power supplies that supply power to a platform.

Register Grouping

Consists of two register blocks (it has two pointers to two different blocks of registers). The fixed-position bits within a register grouping can be split between the two register blocks. This allows the bits within a register grouping to be split between two chips.

Reserved Bits

Some unused bits in ACPI hardware registers are designated as “Reserved” in the ACPI specification. For future extensibility, hardware-register reserved bits always return zero, and data writes to them have no side effects. OSPM implementations must write zeros to all reserved bits in enable and status registers and preserve bits in control registers.

Root System Description Pointer (RSDP)

An ACPI-compatible system must provide an RSDP in the system’s low address space. This structure’s only purpose is to provide the physical address of the RSDT and XSDT.

Root System Description Table (RSDT)

A table with the signature ‘RSDT,’ followed by an array of physical pointers to other system description tables. The OS locates that RSDT by following the pointer in the RSDP structure.

Runtime Firmware

Generic term to describe any firmware on a platform used during runtime (i.e., after the boot process has completed). Use a more specific term, if possible.

Secondary System Description Table (SSDT)

SSDTs are a continuation of the DSDT. Multiple SSDTs can be used as part of a platform description. After the DSDT is loaded into the ACPI Namespace, each secondary description table listed in the RSDT/XSDT with a unique OEM Table ID is loaded. This allows the OEM to provide the base support in one table, while adding smaller system options in other tables.

System Physical Address (SPA)

The platform physical address assigned and programmed by the platform and utilized by the OS.

Sleep Button

A user push button that switches the system from the sleeping/soft off state to the working state, and signals the OS to transition to a sleeping state from the working state.

Smart Battery Subsystem

A battery subsystem that conforms to the following specifications: Smart Battery and either Smart Battery System Manager or Smart Battery Charger and Selector—and the additional ACPI requirements.

Smart Battery Table

An ACPI table used on platforms that have a Smart Battery subsystem. This table indicates the energy-level trip points that the platform requires for placing the system into different sleeping states and suggested energy levels for warning the user to transition the platform into a sleeping state.

SMBus Interface

A standard hardware and software communications interface between an OS bus driver and an SMBus controller.

Software

Software is comprised of elements required to load the operating system and all user applications and user data

subsequently handled by the operating system.

System

A system is the entirety of a computing entity, including all elements in a platform (hardware, firmware) and software (operating system, user applications, user data). A system can be thought of both as a logical construct (e.g. a software stack) or physical construct (e.g. a notebook, a desktop, a server, a network switch, etc).

System BIOS

A term sometimes used in industry to refer to either Legacy BIOS, or to UEFI Core System BIOS, or both. Please use this term only when referring to Legacy BIOS. See also: BIOS, Legacy BIOS.

System Context

The volatile data in the system that is not saved by a device driver.

System Control Interrupt (SCI)

A system interrupt used by hardware to notify the OS of ACPI events. The SCI is an active, low, shareable, level interrupt.

System Management Bus (SMBus)

A two-wire interface based upon the I²C protocol. The SMBus is a low-speed bus that provides positive addressing for devices, as well as bus arbitration.

System Management Interrupt (SMI)

An OS-transparent interrupt generated by interrupt events on legacy systems. By contrast, on ACPI systems, interrupt events generate an OS-visible interrupt that is shareable (edge-style interrupts will not work). Hardware platforms that want to support both legacy operating systems and ACPI systems must support a way of re-mapping the interrupt events between SMIs and SCIs when switching between ACPI and legacy models.

Thermal States

Thermal states represent different operating environment temperatures within thermal zones of a system. A system can have one or more thermal zones; each thermal zone is the volume of space around a particular temperature-sensing device. The transitions from one thermal state to another are marked by trip points, which are implemented to generate an SCI when the temperature in a thermal zone moves above or below the trip point temperature.

UEFI

One form of Host Processor Boot Firmware which uses a Unified Extensible Firmware Interface (UEFI) structure (as defined by the UEFI Forum). This is the current host processor boot firmware structure being adopted as a standard in the industry. This term should be used when referring specifically to UEFI code on a platform.

UEFI Drivers

Standalone binary executables in PE/COFF format which are loaded by UEFI during the boot process to handle specific pieces of hardware.

eXtended Root System Description Table (XSDT)

The XSDT provides identical functionality to the RSDT but accommodates physical addresses of DESCRIPTION HEADERS that are larger than 32 bits. Notice that both the XSDT and the RSDT can be pointed to by the RSDP structure.

2.2 Global System State Definitions

Global system states (Gx states) apply to the entire system and are visible to the user.

Global system states are defined by six principal criteria:

1. Does application software run?
2. What is the latency from external events to application response?
3. What is the power consumption?
4. Is an OS reboot required to return to a working state?
5. Is it safe to disassemble the computer?
6. Can the state be entered and exited electronically?

Following is a list of the system states:

G3 Mechanical Off

A computer state that is entered and left by a mechanical means (for example, turning off the system's power through the movement of a large red switch). It is implied by the entry of this off state through a mechanical means that no electrical current is running through the circuitry and that it can be worked on without damaging the hardware or endangering service personnel. The OS must be restarted to return to the Working state. No hardware context is retained. Except for the real-time clock, power consumption is zero.

G2/S5 Soft Off

A computer state where the computer consumes a minimal amount of power. No user mode or system mode code is run. This state requires a large latency in order to return to the Working state. The system's context will not be preserved by the hardware. The system must be restarted to return to the Working state. It is not safe to disassemble the machine in this state.

G1 Sleeping

A computer state where the computer consumes a small amount of power, user mode threads are not being executed, and the system "appears" to be off (from an end user's perspective, the display is off, and so on). Latency for returning to the Working state varies on the wake environment selected prior to entry of this state (for example, whether the system should answer phone calls). Work can be resumed without rebooting the OS because large elements of system context are saved by the hardware and the rest by system software. It is not safe to disassemble the machine in this state.

G0 Working

A computer state where the system dispatches user mode (application) threads and they execute. In this state, peripheral devices (peripherals) are having their power state changed dynamically. The user can select, through some UI, various performance/power characteristics of the system to have the software optimize for performance or battery life. The system responds to external events in real time. It is not safe to disassemble the machine in this state.

S4 Non-Volatile Sleep

A special global system state that allows system context to be saved and restored (relatively slowly) when power is lost to the motherboard. If the system has been commanded to enter S4, the OS will write all system context to a file on non-volatile storage media and leave appropriate context markers. The machine will then enter the S4 state. When the system leaves the Soft Off or Mechanical Off state, transitioning to Working (G0) and restarting the OS, a restore from a NVS file can occur. This will only happen if a valid non-volatile sleep data set is found, certain aspects of the configuration of the machine have not changed, and the user has not manually aborted the restore. If all these conditions are met, as part of the OS restarting, it will reload the system context and activate it. The net effect for the user is what looks like a resume from a Sleeping (G1) state (albeit slower). The aspects of the machine configuration that must not change include, but are not limited to, disk layout and memory size. It might be possible for the user to swap a PC Card or a Device Bay device, however.

Notice that for the machine to transition directly from the Soft Off or Sleeping states to S4, the system context must be written to non-volatile storage by the hardware; entering the Working state first so that the OS or platform runtime firmware can save the system context takes too long from the user's point of view. The transition from Mechanical Off to S4 is likely to be done when the user is not there to see it.

Because the S4 state relies only on non-volatile storage, a machine can save its system context for an arbitrary period of time (on the order of many years).

Table 2.1: Summary of Global Power States

Global state	system	Software runs	Latency	Power consumption	OS restart required	Safe to dis-assemble computer	Exit state electronically
G0 Working		Yes	0	Large	No	No	Yes
G1 Sleeping		No	>0, varies with sleep state	Smaller	No	No	Yes
G2/S5 Soft Off		No	Long	Very near 0	Yes	No	Yes
G3 Mechanical Off		No	Long	RTC battery	Yes	Yes	No

Notice that the entries for G2/S5 and G3 in the Latency column of the above table are “Long.” This implies that a platform designed to give the user the appearance of “instant-on,” similar to a home appliance device, will use the G0 and G1 states almost exclusively (the G3 state may be used for moving the machine or repairing it).

2.3 Device Power State Definitions

Device power states are states of particular devices; as such, they are generally not visible to the user. For example, some devices may be in the Off state even though the system as a whole is in the Working state.

Device states apply to any device on any bus. They are generally defined in terms of four principal criteria:

- Power consumption—How much power the device uses.
- Device context—How much of the context of the device is retained by the hardware. The OS is responsible for restoring any lost device context (this may be done by resetting the device).
- Device driver—What the device driver must do to restore the device to full on.
- Restore time—How long it takes to restore the device to full on.

The device power states are defined below, although very generically. Many devices do not have all four power states defined. Devices may be capable of several different low-power modes, but if there is no user-perceptible difference between the modes, only the lowest power mode will be used. The Device Class Power Management Specifications, included in Appendix A of this specification, describe which of these power states are defined for a given type (class) of device and define the specific details of each power state for that device class. For a list of the available Device Class Power Management Specifications, see [Appendix A: Device Class Specifications](#).

D3 (Off)

Power has been fully removed from the device. Also referred to as D3cold in this and other specs. All device context is lost when this state is entered, so the OS software will reinitialize the device when powering it back on. Since all device context and power are lost, devices in this state do not decode their address lines, and cannot be enumerated by software. Devices in this state have the longest restore times.

D3hot

The meaning of the D3hot State is defined by each device class. In general, D3hot is expected to save as much power as possible without affecting PNP Enumeration. Devices in D3hot must have enough power to remain

enumerable by software. For example, PCI Configuration space access and contents must operate as in shallower power states. Similarly, ACPI identification and configuration objects must operate as in shallower power states. Otherwise, no device functionality is supported, and Driver software is required to restore any lost context, or reinitialize the device, during its transition back to D0.

Devices in this state can have long restore times. All classes of devices define this state.

Note

For devices that support both D3hot and D3 exposed to OSPM via _PR3, device software/drivers must always assume OSPM will target D3 and must assume all device context will be lost and the device will no longer be enumerable.

D2

The meaning of the D2 Device State is defined by each device class. Many device classes may not define D2. In general, D2 is expected to save more power and preserve less device context than D1 or D0. Buses in D2 may cause the device to lose some context (for example, by reducing power on the bus, thus forcing the device to turn off some of its functions).

D1

The meaning of the D1 Device State is defined by each device class. Many device classes may not define D1. In general, D1 is expected to save less power and preserve more device context than D2.

D0 (Fully-On)

This state is assumed to be the highest level of power consumption. The device is completely active and responsive, and is expected to remember all relevant context continuously.

Transitions amongst these power states are restricted for simplicity. Power-down transitions (from higher-power, or shallower, to lower-power, or deeper) are allowed between any two states. However, power-up transitions (from deeper to shallower) are required to go through D0; i.e. D_y to D_x<y is illegal for all x !=0.

Table 2.2: Summary of Device Power States

Device State	Power Consumption	Device Context Retained	Driver Restoration
D0 - Fully-On	As needed for operation	All	None
D1	D0>D1>D2> D3hot>D3	>D2	<D2
D2	D0>D1>D2> D3hot>D3	<D1	>D1
D3hot	D0>D1>D2>D3hot>D3	Optional	None <->Full initialization and load
D3 - Off	0	None	Full initialization and load

Note

Devices often have different power modes within a given state. Devices can use these modes as long as they can automatically transparently switch between these modes from the software, without violating the rules for the current D_x state the device is in. Low-power modes that adversely affect performance (in other words, low speed modes) or that are not transparent to software cannot be done automatically in hardware; the device driver must issue commands to use these modes.

2.3.1 Device Performance States

Device performance states (Px states) are power consumption and capability states within the active (D0) device power state. Performance states allow OSPM to make tradeoffs between performance and energy conservation. Device performance states have the greatest impact when the implementation is such that the states invoke different device efficiency levels as opposed to a linear scaling of performance and energy consumption. Since performance state transitions occur in the active device states, care must be taken to ensure that performance state transitions do not adversely impact the system.

Device performance states, when necessary, are defined on a per device class basis (See [Appendix A: Device Class Specifications](#) for more information).

2.4 Sleeping and Soft-off State Definitions

S1-S4 are types of sleeping states within the global system state, G1, while S5 is a soft-off state associated with the G2 system state. The Sx states are briefly defined below.

For a detailed definition of the system behavior within each Sx state, see [_Sx \(System States\)](#). For a detailed definition of the transitions between each of the Sx states, see [Sleeping States](#).

S1 Sleeping State

The S1 sleeping state is a low wake latency sleeping state. In this state, no system context is lost (CPU or chip set) and hardware maintains all system context.

S2 Sleeping State

The S2 sleeping state is a low wake latency sleeping state. This state is similar to the S1 sleeping state except that the CPU and system cache context is lost (the OS is responsible for maintaining the caches and CPU context). Control starts from the processor's reset vector after the wake event.

S3 Sleeping State

The S3 sleeping state is a low wake latency sleeping state where all system context is lost except system memory. CPU, cache, and chip set context are lost in this state. Hardware maintains memory context and restores some CPU and L2 configuration context. Control starts from the processor's reset vector after the wake event.

S4 Sleeping State

The S4 sleeping state is the lowest power, longest wake latency sleeping state supported by ACPI. In order to reduce power to a minimum, it is assumed that the hardware platform has powered off all devices. Platform context is maintained.

S5 Soft Off State

The S5 state is similar to the S4 state except that the OS does not save any context. The system is in the "soft" off state and requires a complete boot when it wakes. Software uses a different state value to distinguish between the S5 state and the S4 state to allow for initial boot operations within the platform boot firmware to distinguish whether the boot is going to wake from a saved memory image.

2.5 Processor Power State Definitions

Processor power states (Cx states) are processor power consumption and thermal management states within the global working state, G0. The Cx states possess specific entry and exit semantics and are briefly defined below. For a more detailed definition of each Cx state, see [Processor Power States](#).

C0 Processor Power State

While the processor is in this state, it executes instructions.

C1 Processor Power State

This processor power state has the lowest latency. The hardware latency in this state must be low enough that the operating software does not consider the latency aspect of the state when deciding whether to use it. Aside from putting the processor in a non-executing power state, this state has no other software-visible effects.

C2 Processor Power State

The C2 state offers improved power savings over the C1 state. The worst-case hardware latency for this state is provided via the ACPI system firmware and the operating software can use this information to determine when the C1 state should be used instead of the C2 state. Aside from putting the processor in a non-executing power state, this state has no other software-visible effects.

C3 Processor Power State

The C3 state offers improved power savings over the C1 and C2 states. The worst-case hardware latency for this state is provided via the ACPI system firmware and the operating software can use this information to determine when the C2 state should be used instead of the C3 state. While in the C3 state, the processor's caches maintain state but ignore any snoops. The operating software is responsible for ensuring that the caches maintain coherency.

2.6 Device and Processor Performance State Definitions

Device and Processor performance states (Px states) are power consumption and capability states within the active/executing states, C0 for processors and D0 for devices. The Px states are briefly defined below. For a more detailed definition of each Px state from a processor perspective, see *Processor Performance Control*. For a more detailed definition of each Px state from a device perspective see *Device and Processor Performance States*, and *Appendix A: Device Class Specifications*.

P0 Performance State

While a device or processor is in this state, it uses its maximum performance capability and may consume maximum power.

P1 Performance State

In this performance power state, the performance capability of a device or processor is limited below its maximum and consumes less than maximum power.

Pn Performance State

In this performance state, the performance capability of a device or processor is at its minimum level and consumes minimal power while remaining in an active state. State n is a maximum number and is processor or device dependent. Processors and devices may define support for an arbitrary number of performance states not to exceed 255.